Quiz (on Gradescope) (10:30-10:40)

Quiz (on Gradescope) (10:30-10:40)

```
main():
    h1, c1 = spawn(my_func)
    h2, c2 = spawn(my_func)
    send(c2, null)
    print("1")
    join(h1)
    join(h2)
    print("0")
```

```
my_func(c):
    recv(c)
    print("2")
```

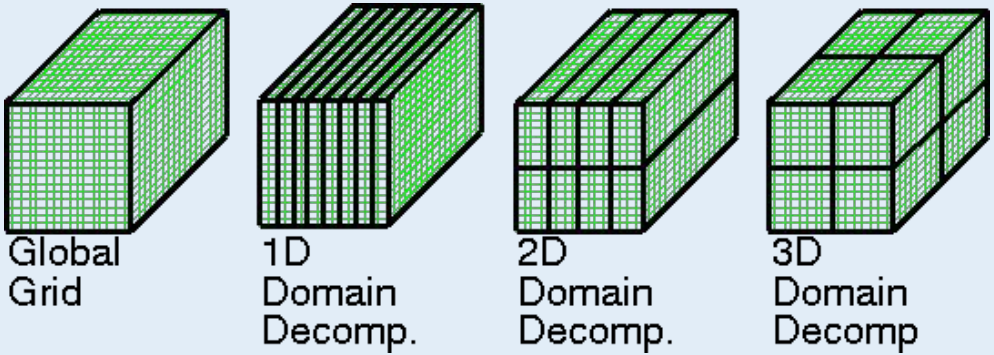
What are all possible outputs?

Hint: exit() is called if all routines are blocked.

spawn, yield, send, recv, join **can** all switch (not **must**)

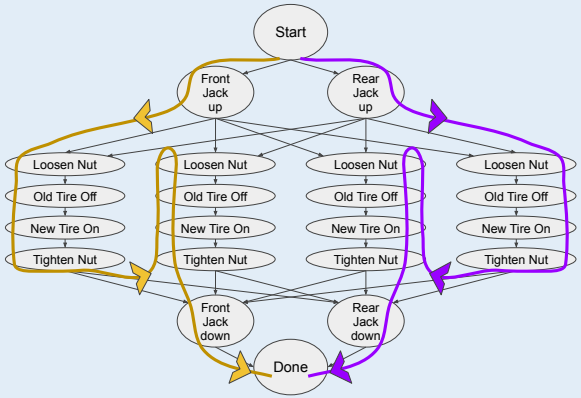
Decomposition

CSE 493L



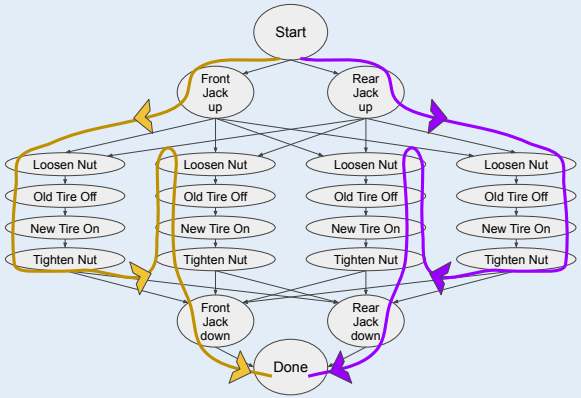
Concurrency vs. Parallelism

- Concurrency
  - Property of the dependency graph
- Parallelism
  - Property of an execution of a dependency graph



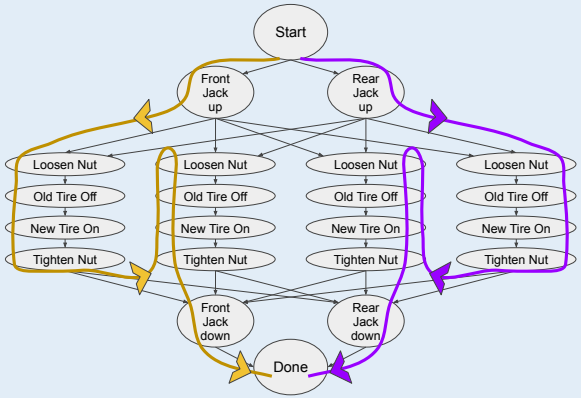
Concurrency vs. Parallelism

- Concurrency
  - Multiple things that could be done
- Parallelism
  - Multiple things being done at the same time



Concurrency vs. Parallelism

- Concurrency
  - A programming paradigm
- Parallelism
  - A difficult performance optimization
  - Only possible with concurrency “exposed”



How do you expose concurrency?

- Create programs that scale with the number of workers
- Create many mini-programs that workers can specialize on

How do you expose concurrency?

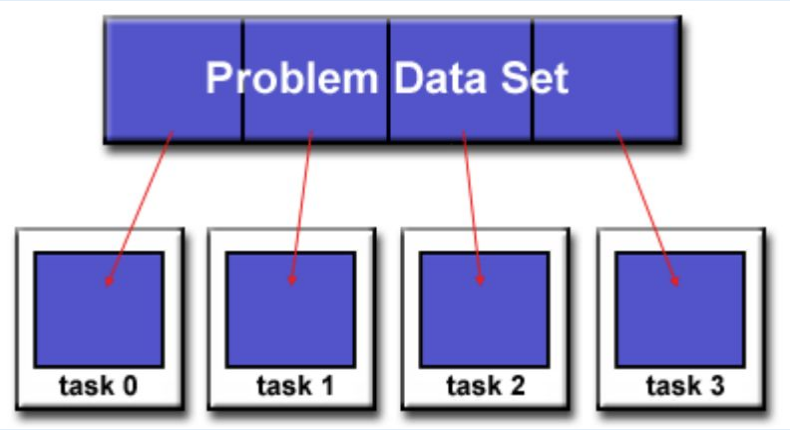
- Allow for concurrency across multiple instances of your program
- Allow for concurrency within one instance of your program

How do you expose concurrency?

- Domain decomposition
- Functional decomposition

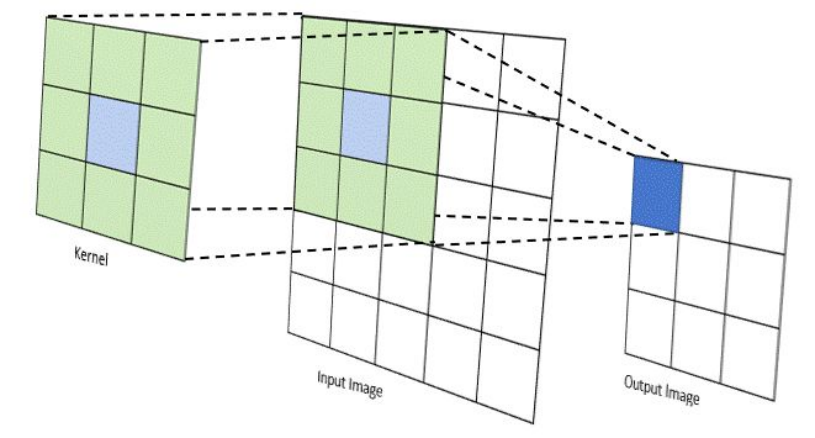
Domain Decomposition

- Can be input, output, or both
- sometimes “embarrassingly parallel”
- e.g. add 1 to every integer in array
- Each worker does the same thing



Example: Image Blurring

```
output[i, j] = mean(all of input[i, j]'s neighbors)
```



Domain Decomposition of Image Blurring

Each worker computes part of the output space

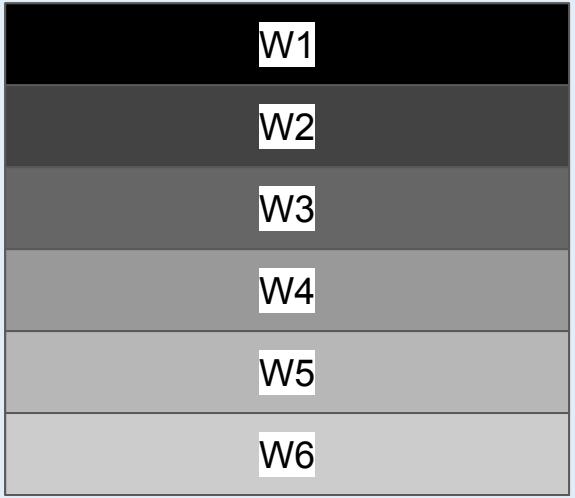
- Some workers overlap data!
- What if each worker runs at a different speed?
  - “partitioning” and “load balancing” questions
- What if you have 100,000 CPUs? (aka a GPU)

```
output[i, j] = mean(all of input[i, j]’s neighbors)
```

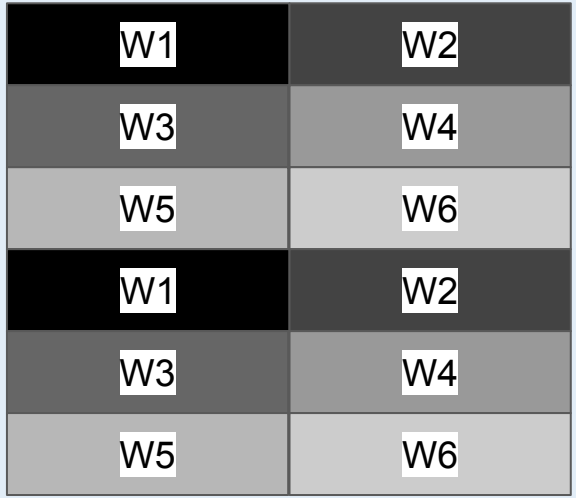
Domain Decomposition - Static Allocation

- Each worker has some tasks. Which tasks?

“Block Allocation”



“Block-Cyclic Allocation”



Domain Decomposition - Dynamic Allocation

- Each worker finishes at a different speed
- Slowest worker determines performance
- You'd like to balance work dynamically between workers
  - But "balance" implies time spent communicating rather than working!

Domain Decomposition - Load Balancing Example

If we split up the problem into 2 giant tasks, then 1 worker finishing ahead of the other is bad

worker 0 worker 1

Coarse-Grained Parallelism

Domain Decomposition - Load Balancing Example

If we split up the problem into 1000 mini tasks, then we have to spend time communicating about who’s doing what task

0 1 2 3 4 5 6

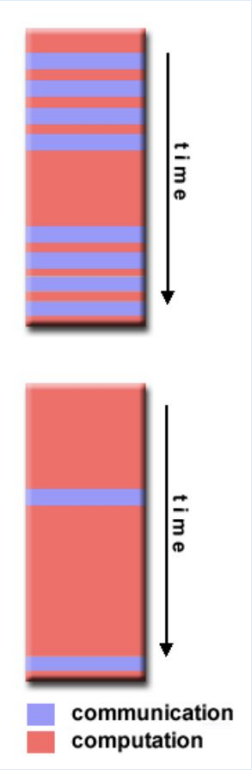
Fine-Grained Parallelism

Domain Decomposition - Load Balancing Granularity

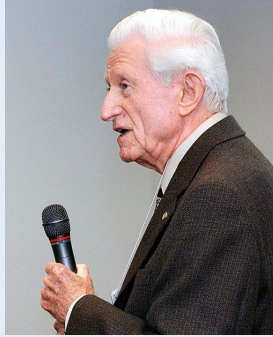
$$G = \frac{Computation}{Communication}$$

- Fine-grained parallelism
  - Good, frequent load balancing
  - G is small -> potentially high overhead
  - Hard to get right!

- Coarse-grained parallelism
  - Limited load balancing
  - G is big -> low overhead
  - Easier to get right

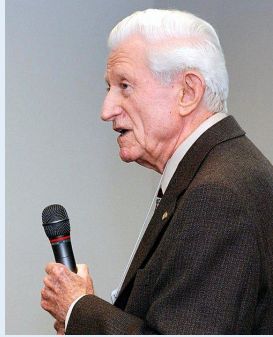


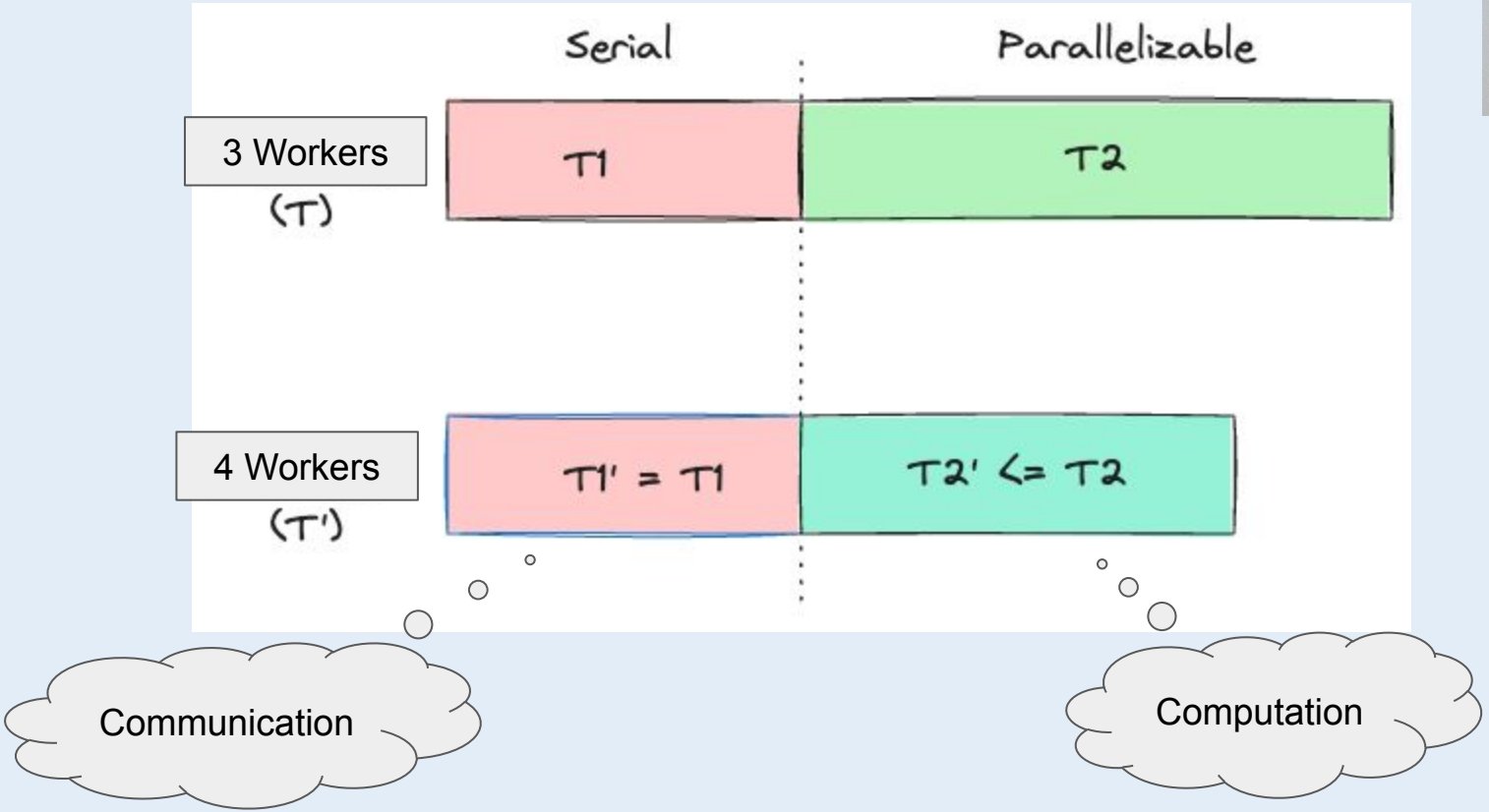
Domain Decomposition - Amdahl’s Law



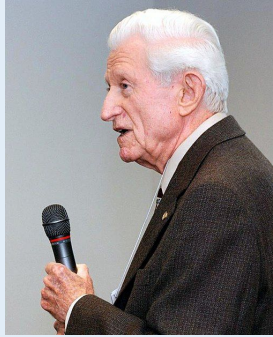
- Some parts of a routine (code) are inherently sequential
  - Some parts do not speed up when you add more workers
- e.g. loading data, load-balancing overheads/communication, …
- Amdahl’s Law describes how scaling up resource count affects runtime

Amdahl's Law, Imprecisely



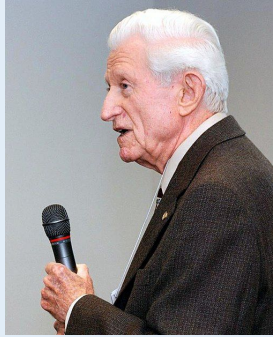


Amdahl’s Law



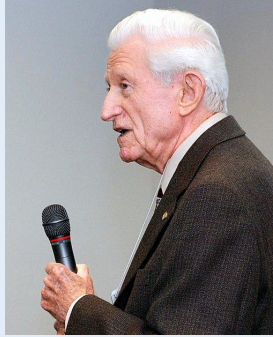
- Example program:
  - Read 100GB of numbers from disk (1 min)
  - Square each number (2 min)

Amdahl’s Law



- Example program with 2 cores:
  - Read 100GB of numbers from disk (1 min)
  - Square each number (1 min)

Amdahl’s Law

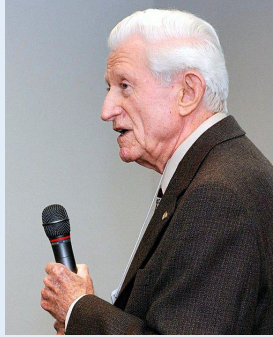


- Example program with 2 cores:
  - Read 100GB of numbers from disk (1 min)
  - Square each number (1 min)
- Speedup = T(1 core) / T(2 cores)

  = 3 min / 2 min

  = 1.5x

Amdahl's Law



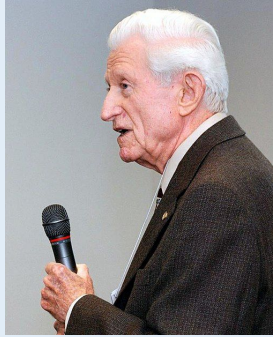
- Example program with 2 cores:
  - Read 100GB of numbers from disk (1 min)
  - Square each number (1 min)
- Speedup = T(1 core) / T(2 cores)

  = 3 min / 2 min

  = 1.5x

  … even though we doubled our cores, we only get 1.5x performance!

Amdahl's Law, Precisely



Let $\mathrm{T(N, W)}$ be the time taken to complete W workloads with N cores.

Let t be the time taken to complete one workload with one core.

Let p be the proportion of t that is parallelizable

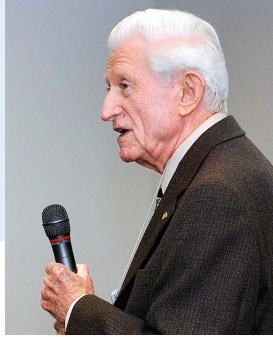
In other words, $\mathrm{T(1, 1)} = \mathrm{t} = t * p + t * (1 - p)$

I claim that $\mathrm{T(1, N)} = \dfrac{t * p}{N} + t * (1 - p)$

And $\mathrm{T(W, N)} = \mathrm{W} * \mathrm{T(1, N)}$

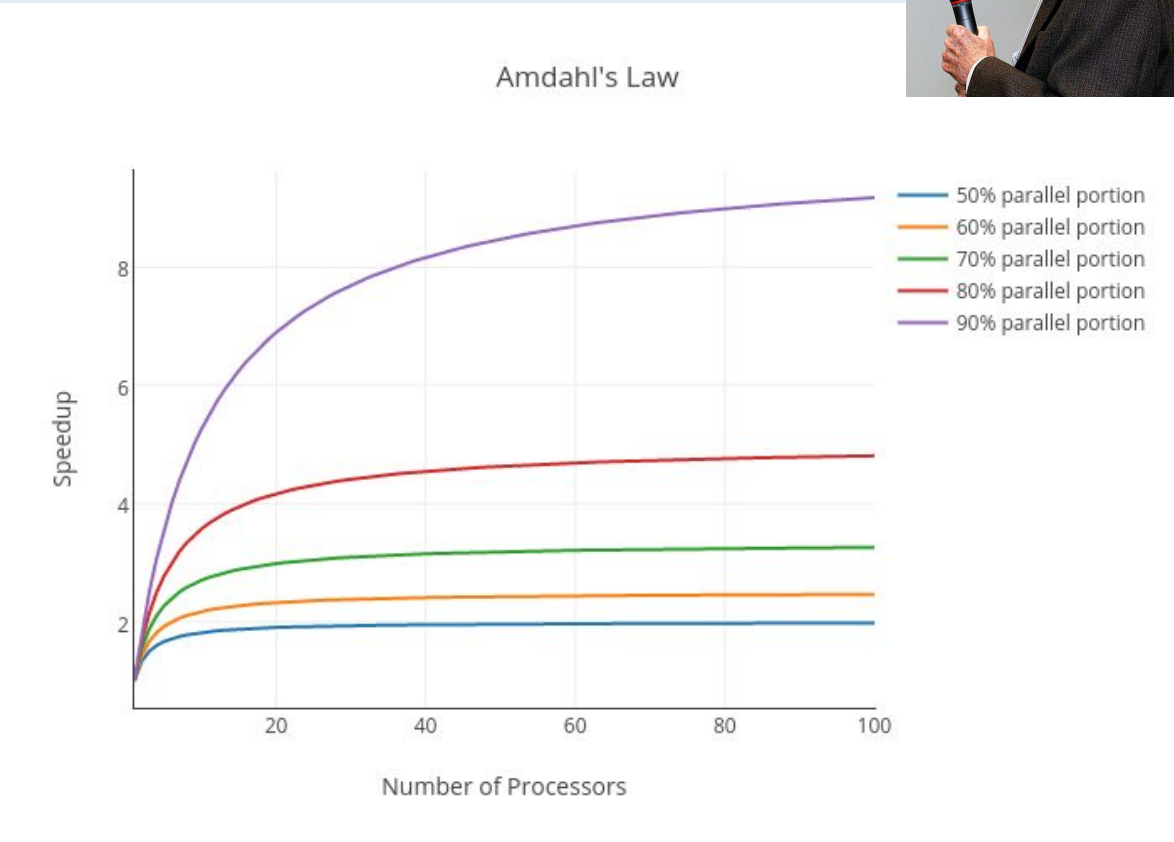
So, Speedup = $\mathrm{T(W, 1)}$ / $\mathrm{T(W, N)} = \dfrac{1}{\frac{p}{N} + (1 - p)}$

Amdahl’s Law, Precisely



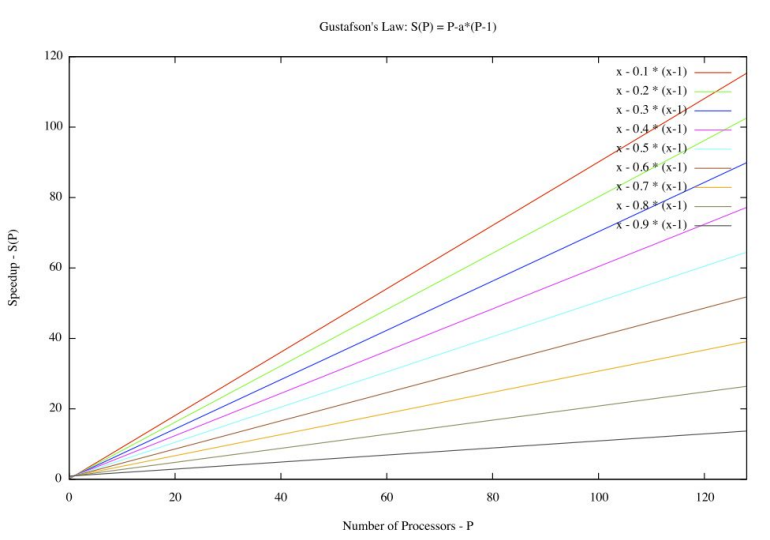
- If your code is only 50% parallel, you will never get more than a 2x speedup even with unlimited cores.

$$\frac{1}{\frac{p}{N} + (1 - p)}$$

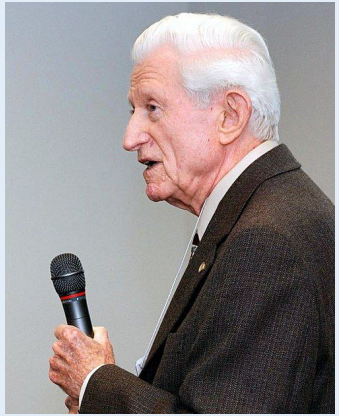


Gustafson’s Law

- Gustafson argued that Amdahl’s Law is not realistic
  - Instead of halving the runtime, we can double the work for a 2x speedup!
  - In many problems, the Workload (W) increases when you add more cores
  - People keep the amount of work per CPU fixed in practice
- If W increases linearly, then Speedup increases linearly



Who is right?



Who is right?

It depends!

Running a videogame -> Amdahl’s Law

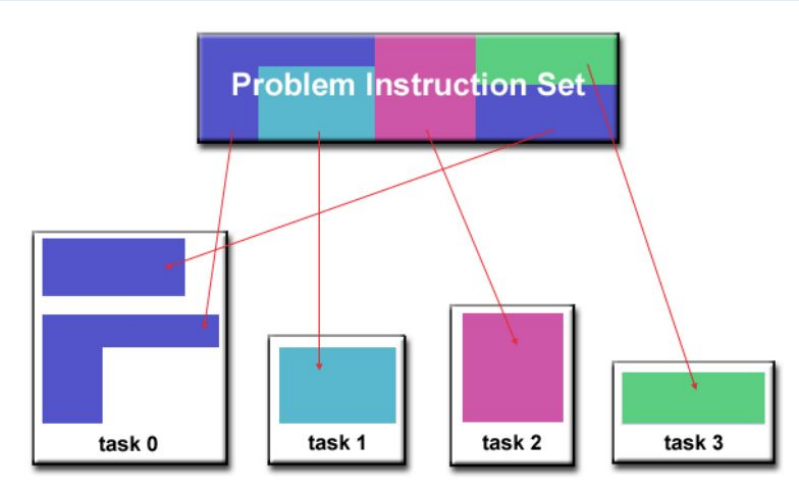
Training a big ML model -> Gustafson’s Law

What are the general techniques to parallelize a program?

- Domain decomposition
- Functional decomposition

Functional Decomposition

- Divide up the “instructions” / routines
  - Dependency graphs!
- Typically more difficult
  - requires data movement
- aka “pipeline parallelism”
- e.g. pipelines / dataflows

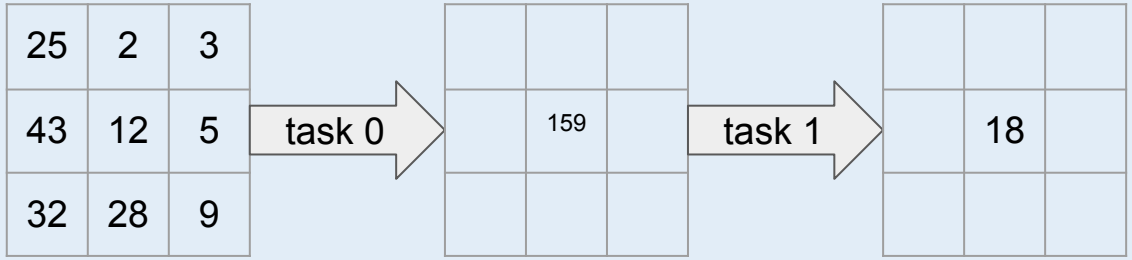


Functional Decomposition of Image Blurring

Program: mean = sum / count

One task computes the sum

One task comes in after and divides by 9




```
output[i, j] = mean(all of input[i, j]'s neighbors)
```

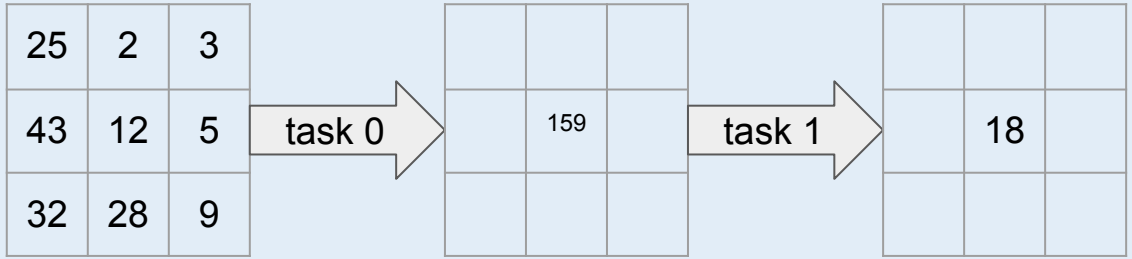
Functional Decomposition of Image Blurring

Program: mean = sum / count

One task computes the sum

One task comes in after and divides by 9

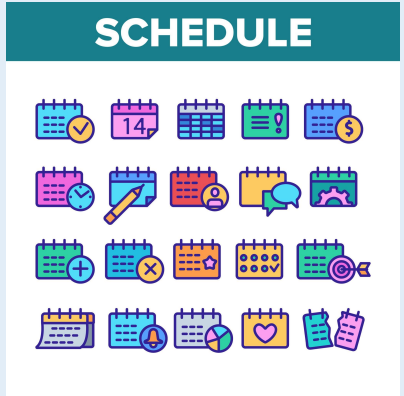
- What if you have more than 2 cores?




```
output[i, j] = mean(all of input[i, j]'s neighbors)
```

Functional Decomposition - Schedules

- Data “flows” through from one worker to the next
- We must assign when that happens, and deal with realities of slow workers!



Functional Decomposition - Schedules

The ideal:

Time (s)	Worker 0	Worker 1
0	Pixel 0	<wait>
1	Pixel 1	Pixel 0
2	Pixel 2	Pixel 1
3	Pixel 3	Pixel 2
4	…	…

Functional Decomposition - Schedules

In practice:

Time (s)	Worker 0	Worker 1
0	Pixel 0	<wait>
1	Pixel 1	
2		Pixel 0
3	Pixel 2	
4	Pixel 3	Pixel 1
5		Pixel 2
6	…	…

Functional Decomposition - Schedules

In practice:

Time (s)	Worker 0	Worker 1
0	Pixel 0	<wait>
1	Pixel 1	Pixel 0
2	…	Pixel 1
3	Pixel N (done)	…
4		Pixel N-1
5		Pixel N
6		

Functional Decomposition - Schedules

With communication:

Time (s)	Worker 0	Worker 1
0	Pixel 0	
1	Pixel 1…	Pixel 0
2	Pixel N	Pixel 1…
3	**<forced wait>**	Pixel N
4	Pixel 0	<wait>
5	Pixel 1	Pixel 0
6	Pixel 2…	Pixel 1…

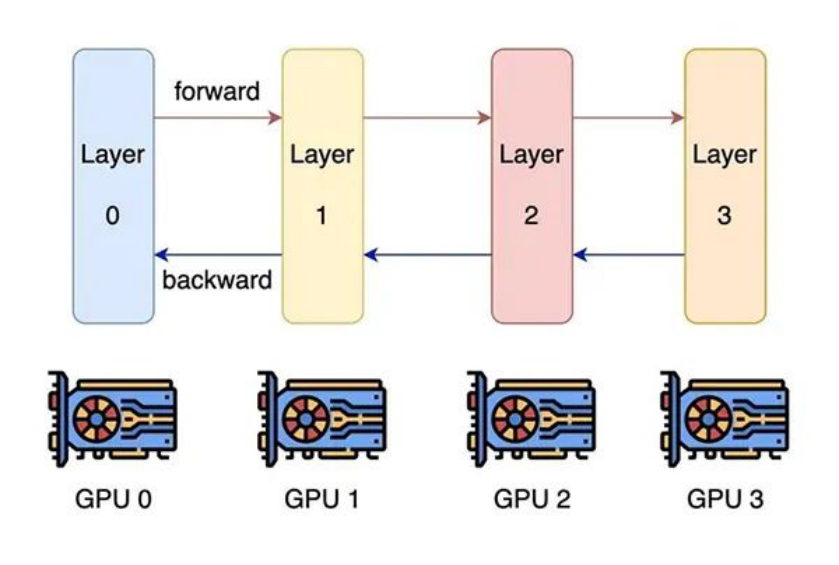
How do you know if domain or functional decomposition is faster?

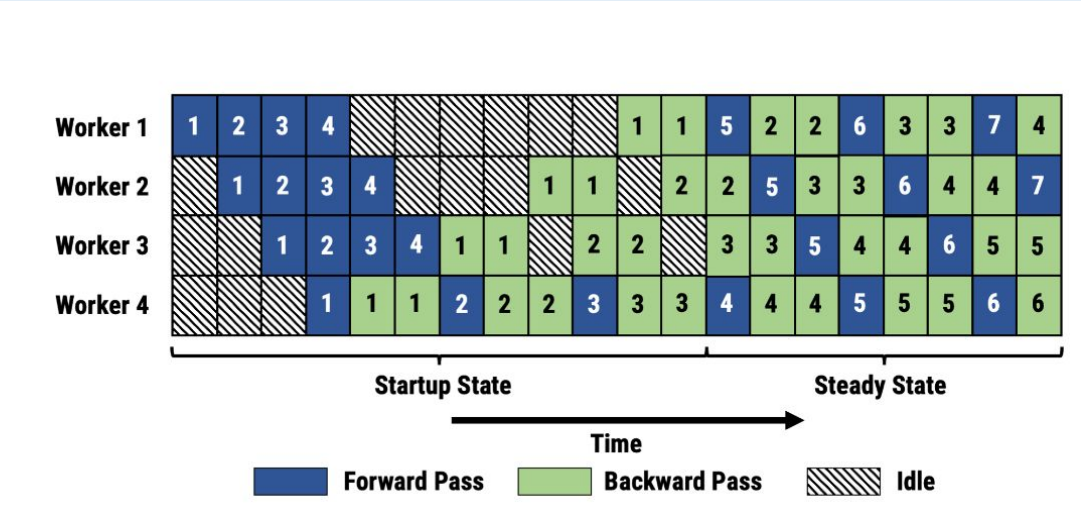
These strategies are not mutually exclusive

- Domain decomposition and functional decomposition are better together
- e.g. blurring 1,000 images
  - can use functional decomposition within the images and domain decomposition across them, or vice versa
  - can use both within the images and both across them

Real world example of functional & domain decomposition

- Machine Learning





Measuring Speedup

Baseline: **best** serial algorithm

Example: Parallel Bubble Sort

- Serial: 150s
- Parallel: 40s
- Speedup: 3.75x !?

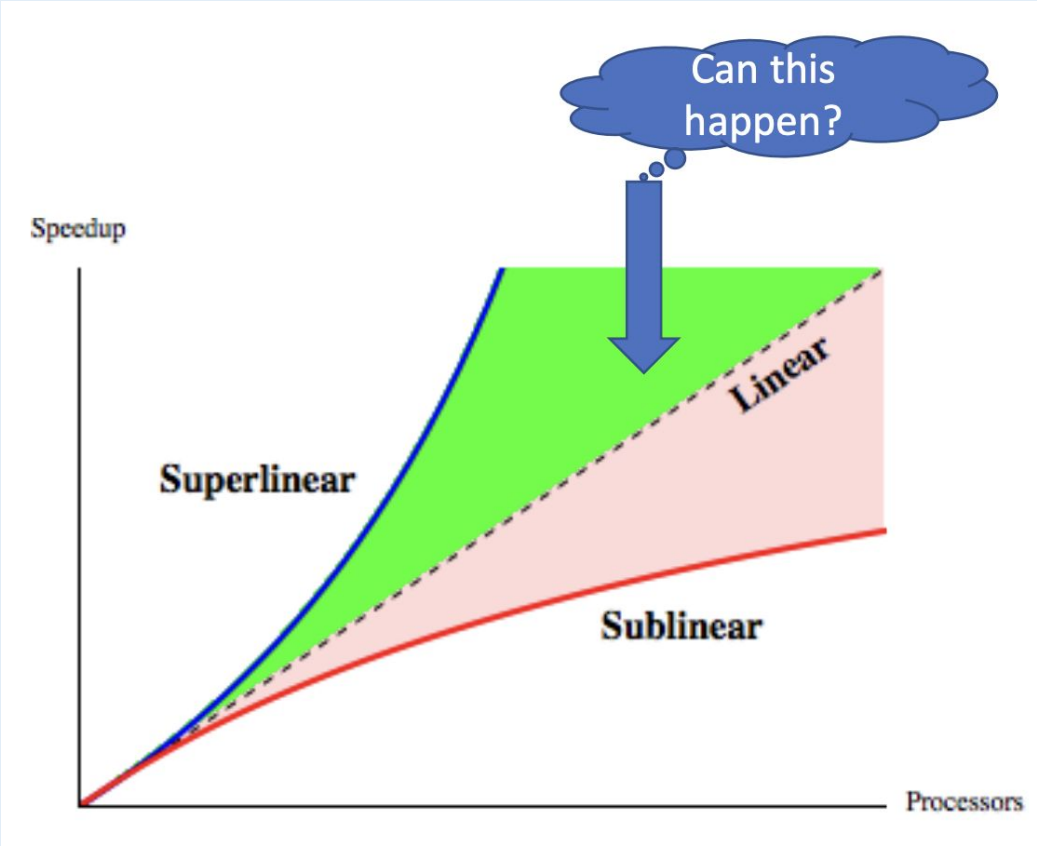
NO

- Serial Quicksort: 30s
- Speedup: 0.75x

Why do I insist on best serial algorithm as baseline?

Measuring Speedup

- Is linear speedup possible?
- Is sublinear speedup possible?
- Is superlinear speedup possible?
  - Possibly due to cache
  - Probably poor methodology



To Recap:

Concurrency ≠ Parallelism

Exposing more concurrency often leads to parallelism opportunities

- domain decomposition -> scaling challenges
- functional decomposition -> scheduling challenges